

Bonus Topics – Implementation Support Guide

This document provides additional guidance and example resources for implementing the three bonus topics in the Final Project. The provided examples are optional; you may freely use any reasonable method to complete each topic.

Topic 1: AI Picture Generation

If your computer has strong hardware (GPU / high-performance CPU), you may use more advanced image generation models from HuggingFace that require local deployment. These models typically produce high-quality results but require significant computational resources.

If you prefer a lightweight and simple solution, you may use **Pollinations.ai**, an open and free platform that provides easy-to-use text and image generation APIs.

Pollinations requires **no registration** and **no API key**, making it ideal for simple projects.

You may test the example code we provided to generate images (you may also use this code directly in your Final Project):

Example Code (Google Drive):

https://drive.google.com/file/d/1hKSn2X23UZg09QWa9Q2cGhkaTkoch_bh/view?usp=drive_link

For higher-quality AI-generated images, you may consider using **Replicate**, a cloud-based machine learning model hosting service that allows users to run image generation models directly via API.

Replicate Example Code:

https://drive.google.com/file/d/1tGKNmUFXla8dmHbBPQ7v8VwYWhBdoDcs/view?usp=drive_link

To use Replicate, please note the following:

1. Install required dependencies using pip:

```
pip install replicate pillow requests
```

2. Register an account at: <https://replicate.com/>

3. Bind a valid payment method (an international credit card or Chinese credit card generally works).

You typically only need to deposit **\$1 USD** to activate API usage.

4. Go to the “API Tokens” section on your Replicate account page, create a new token, and paste it into the code in the correct location (usually in the form r8_XXXXXX).
5. After setup, you can run the code to generate images based on your prompts.

Topic 2: NLP Chat Keyword Extraction or Summary

For this topic, you may continue using the LLM from your chatbot and implement summarization or keyword extraction through commands or GUI buttons.

Alternatively, you may use Python NLP libraries to perform keyword extraction or summarization locally.

We provide a starter code that uses:

- **YAKE** for keyword extraction
- **Sumy (Luhn Summarizer)** for chat summarization

Example Code (Google Drive):

https://drive.google.com/file/d/1cKjDHgQmJMLBiN5hUorMwHqVt_UmmOqi/view?usp=drive_link

To run the example code, install the required libraries:

```
pip install yake sumy nltk
```

This code is only a reference implementation. You may modify it or use entirely different tools (e.g., spaCy, TF-IDF, OpenAI/DeepSeek APIs, etc.).

Topic 3: Sentiment Analysis (Emotion Detection)

For emotion detection, you may again use your chatbot’s LLM to analyze the sentiment of each message.

However, if you prefer an offline solution, you may use a lightweight Python library such as **TextBlob**.

We have provided a simple example using TextBlob that returns sentiment labels along with an emoji (e.g., Positive 😊, Neutral 😐, Negative 😡).

Example Code (Google Drive):

[https://drive.google.com/file/d/1mNwk2Qw0kKB2prK1H-ZG0hdM2d7boUYI/view?usp=drive link](https://drive.google.com/file/d/1mNwk2Qw0kKB2prK1H-ZG0hdM2d7boUYI/view?usp=drive_link)

To run the example:

```
pip install textblob nltk
```

TextBlob is simple, fast, and works entirely offline, making it suitable for most student implementations.

Final Notes

The example code provided for all three topics is **for reference only**.

You are free to:

- directly use the provided code,
- modify or extend it,
- or implement your own solution using other tools or APIs.

Any reasonable approach is acceptable as long as the feature is functional and integrated into your chat system.